Embedded Lighting Classification Using Quantized Density Maps

Ali Hussain

October 2026

Abstract

This project implements a four-class lighting classifier on an ESP32 using real UV and IR sensor measurements. The dataset contains 2400 labelled samples from LED, fluorescent, shade, and sunlight conditions. DBSCAN was used to discover groups in the sensor data without using the labels, and Gaussian kernel density estimation (KDE) was then used to model each group.

The KDE results were converted into four 16×16 byte lookup tables. The final stored model data requires 1040 bytes and has a measured worst-case quantization error of 0.193%.

The model was deployed to an ESP32 and tested again using a separately collected batch of 2324 readings. The ESP32 and desktop implementations agreed on every reading in this batch. LED, fluorescent, and shade remained correctly classified, while an overcast outdoor batch was rejected as UNKNOWN because it differed from the clear-sun training measurements.

The final ESP32 program used 308800 bytes of flash and 23740 bytes of SRAM. Classification required approximately 3–6 μs per query.

1 System Design

The goal of the project was to move the density-based classifier from offline Python testing to a working embedded system.

Training is performed on a computer. The ESP32 stores only the final lookup model and the values needed to repeat the same preprocessing.

The two input features are

- an analog UV measurement,
- the IR channel of a TSL2591 light sensor.

The embedded path is

$$\text{read} \rightarrow \text{EMA} \rightarrow \log(1+x) \rightarrow \text{scale} \rightarrow \text{lookup}.$$

DBSCAN and KDE are therefore not executed during normal ESP32 operation.

For every new reading, the two features are converted to a grid location. The ESP32 reads one byte from each class table and selects the class with the largest value.

Algorithm 1 Embedded classification

Require: Sensor readings x, y
Require: Tables Q_c, scaling bounds, G, τ
Ensure: Class or UNKNOWN
Apply EMA filtering
Apply $\log(1+x)$ to both features
Scale using the stored training bounds
Find and clip the grid row and column
for each class c **do**
    $v_c \leftarrow Q_c[row, col]/255$
end for
$\hat{c} \leftarrow \arg\max_c v_c$
if $v_{\hat{c}} < \tau$ **then**
    return UNKNOWN
else
    return $\hat{c}$
end if

With four classes, each prediction requires only four table reads.

2 Hardware and Data

2.1 Prototype

The prototype uses an ESP32 development board, an analog UV sensor, and a TSL2591 light sensor.

The UV sensor was connected to GPIO34.

The TSL2591 used I2C with

$$\text{SDA} = 21, \qquad \text{SCL} = 22.$$

The TSL2591 was configured with medium gain and a 100 ms integration time.

The original assignment diagram showed a separate analog IR detector. The prototype used the IR channel of the TSL2591 instead. This provided a stable IR measurement and allowed the same sensor configuration to be used during data collection and live testing.

Figure 1 shows the completed prototype.

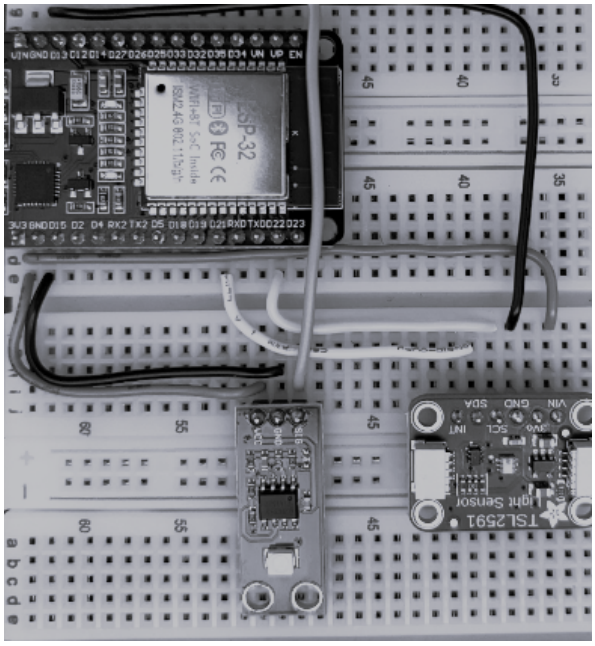


Figure 1: ESP32 prototype with the UV and TSL2591 sensors.

2.2 Data Collection

Measurements were streamed through the Serial Monitor and stored as labelled CSV data.

Four lighting conditions were collected.

Table 1: Collected dataset.

Condition	Samples
LED	600
Fluorescent	600
Shade	600
Sun	600

The total dataset contains

$$4(600) = 2400$$

samples.

2.3 EMA Filtering

An exponential moving average was used to reduce short sensor fluctuations.

The filter is

$$s_n = \alpha x_n + (1 - \alpha) s_{n-1}$$

with

$$\alpha = 0.2.$$

This means that each new filtered value contains 20% of the newest measurement and 80% of the previous filtered value.

EMA was selected because it needs only one previous value and is inexpensive to run continuously on the ESP32.

3 Training and Model Generation

3.1 Data Split

The original dataset was divided using the fixed random seed

$$20250903.$$

The split was

$$60\% \text{ training}, \qquad 20\% \text{ validation}, \qquad 20\% \text{ test}.$$

This produced

$$1440$$

training samples,

$$480$$

validation samples, and

$$480$$

held-out test samples.

Each class contributed 360 training, 120 validation, and 120 test readings.

3.2 Feature Transformation

The sunlight readings were much larger than the indoor readings.

When only min–max scaling was used, the indoor measurements were compressed into a small part of the feature space.

A logarithmic transformation was therefore applied to both sensor features:

$$x' = \log(1 + x).$$

The transformed features were then min–max scaled.

The scaling limits were calculated using the training split only. The same limits were later stored in `model.h` for the ESP32.

3.3 DBSCAN

DBSCAN was run using only the two sensor features. The known lighting labels were not given to the clustering algorithm.

The final settings were

$$\varepsilon = 0.02$$

and

$$\texttt{min_samples} = 8.$$

Four training clusters were found and no training samples were marked as noise.

Table 2: Ground-truth labels compared with DBSCAN clusters.

True label	C0	C1	C2	C3
LED	360	0	0	0
Fluorescent	0	360	0	0
Shade	0	0	360	0
Sun	0	0	0	360

The final mapping was

$$C0 = \text{LED}, \qquad C1 = \text{fluorescent},$$

$$C2 = \text{shade}, \qquad C3 = \text{sun}.$$

The perfect diagonal in Table 2 occurred because the four training groups were compact and well separated after the feature transformation.

The indoor classes were separated mainly by IR level. Sunlight was farther away because it produced a much larger response in both features.

Figure 2 shows the final DBSCAN result.

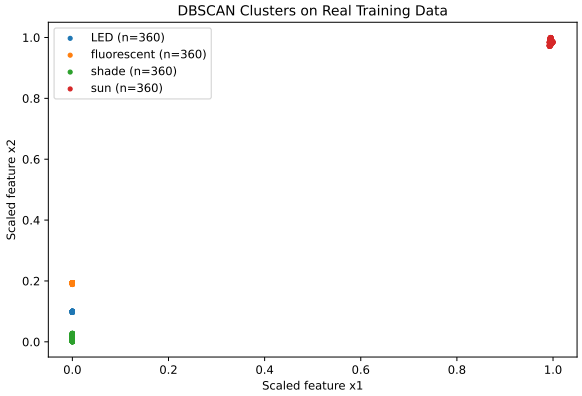


Figure 2: DBSCAN clusters after feature transformation and scaling.

3.4 KDE Bandwidth

A Gaussian KDE was fitted separately to each discovered cluster.

The validation sweep is shown in Table 3.

Table 3: KDE bandwidth validation.

Bandwidth h	Mean log-likelihood
0.03	5.157
0.04	4.660
0.05	4.317
0.06	4.049
0.08	3.626
0.10	3.285
0.12	2.994

The highest score in the tested range occurred at

$$h = 0.03.$$

Figure 3 shows the same sweep.

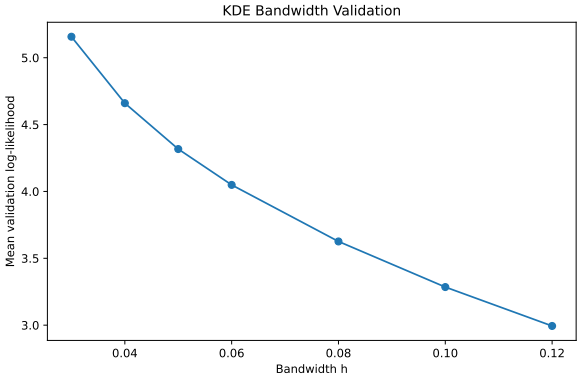


Figure 3: Validation log-likelihood for the tested KDE bandwidths.

The selected value is at the lower end of the sweep. A smaller bandwidth could produce finer density structure, but the embedded model is eventually stored on only a

$$16 \times 16$$

grid.
One grid interval represents

$$\frac{1}{16} = 0.0625$$

of a scaled feature axis. Very small KDE details therefore cannot be preserved exactly by the final stored table.

3.5 Quantized Lookup Model

Each KDE surface was evaluated on a

$$16 \times 16$$

grid and converted to `uint8`.
The four class tables require

$$4(16)(16)(1) = 1024 \text{ bytes.}$$

The four floating-point scaling bounds add

$$4(4) = 16 \text{ bytes.}$$

The total model-data footprint is therefore

$$1024 + 16 = 1040 \text{ bytes.}$$

The measured worst-case quantization error was

$$0.193\%.$$

Table 4: Storage required by the class tables.

Format	Bytes/cell	Total
`uint8`	1	1024 B
`uint16`	2	2048 B
`float32`	4	4096 B
`float64`	8	8192 B

The byte format was selected because it greatly reduced storage while keeping the same class decisions on the original test split.

3.6 Generated Header

The trained model was exported using

`generate_model_header.py`.

The generated `model.h` contains

- `GRID_N = 16`,
- `NUM_CLASSES = 4`,
- the four class tables,
- the four scaling bounds,
- the class names,
- `REJECT_TAU = 0.12`.

4 Experimental Results

4.1 Original Held-Out Test

The first test used the 480 samples that were not used during training or bandwidth selection.

Table 5: Original held-out classification results.

Class	Tested	Correct
LED	120	120
Fluorescent	120	120
Shade	120	120
Sun	120	120
Total	480	480

The resulting accuracy was

$$\frac{480}{480} \times 100 = 100\%.$$

The continuous KDE and the quantized byte-table model also produced the same class decision for all 480 readings.

This result confirms that quantization did not change the decisions on this split. It should not, however, be treated as a complete estimate of real-world accuracy because neighbouring sensor readings from the same recording are highly related.

4.2 Fresh Deployment Batch

A second test was performed using measurements collected separately from the original dataset.

This fresh batch contained

$$581$$

readings for each condition, giving

$$4(581) = 2324$$

new samples.

The same samples were evaluated using both the ESP32 and desktop lookup implementations.

The two implementations agreed on all 2324 readings:

$$\frac{2324}{2324} \times 100 = 100\%.$$

Table 6: Fresh deployment-batch results.

Class	Device=PC	Correct	UNKNOWN
LED	100%	100%	0%
Fluorescent	100%	100%	0%
Shade	100%	100%	0%
Sun	100%	0%	100%

LED, fluorescent, and shade were classified correctly in the fresh batch.

The fresh outdoor readings were collected under overcast conditions, while the original sun training data represented clearer sunlight. All 581 of those readings were rejected as UNKNOWN.

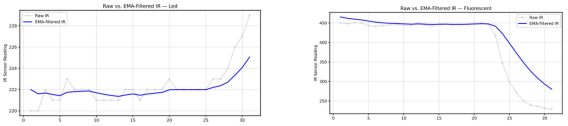
Because the ESP32 and desktop model made the same decision, this was not an ESP32 implementation error. The new outdoor condition simply did not match the stored clear-sun region closely enough.

This also confirms that the reject mechanism is active in the deployed model.

4.3 Filtering Evidence

Raw and EMA-filtered IR readings were recorded during hardware testing.

Two representative sequences are shown in Figure 4.



(a) LED (b) Fluorescent

Figure 4: Measured raw and EMA-filtered IR readings for LED and fluorescent lighting.

The LED sequence contains smaller sample-to-sample changes. The filtered line removes part of this variation while still following the gradual increase near the end of the recording.

The fluorescent sequence contains a much faster decrease in IR level. The EMA follows the same change more slowly, which clearly shows the delay introduced by the filter.

The two examples show the main trade-off of the selected value

$$\alpha = 0.2.$$

Short changes are reduced, but a large real change takes several samples to appear fully in the filtered signal.

The filtered LED and fluorescent levels also remain clearly separated, so the filter does not remove the large difference between these two lighting conditions.

4.4 Model Cost

The expected lookup-table storage was

$$1024 \text{ bytes}.$$

A separate compile was used to isolate the cost of adding the model to the firmware.

The measured flash increase was

1060 bytes.

The difference from the table-only prediction is

$$1060 - 1024 = 36 \text{ bytes.}$$

This corresponds to approximately

$$\frac{36}{1024} \times 100 \approx 3.5\%.$$

The additional space includes the scaling bounds and the small amount of code and model information required to perform the lookup.

No increase in global SRAM was measured when the stored model was added:

$$\Delta\text{SRAM} = 0 \text{ bytes.}$$

The class tables therefore remain in program memory rather than being copied into normal SRAM.

Table 7: Predicted and measured model cost.

Quantity	Prediction	Measurement
Table storage	1024 B	1060 B flash increase
Model SRAM	0 B	0 B increase
Model data	1040 B	1040 B

4.5 Complete Program Memory

The complete final Arduino program used

308800 bytes

of flash and

23740 bytes

of global SRAM.

These totals include the ESP32 core, sensor libraries, Serial code, filtering, and the rest of the firmware, so they are larger than the model-only values in Table 7.

4.6 Inference Time

Classification latency was measured using `micros()`.

The final implementation required approximately

3–6 μs

per classification.

This timing covers the classification operation itself. Sensor integration and Serial printing were not included.

The lookup time is very small compared with the approximately 100 ms sensor sampling period.

5 Discussion

The original test split and the fresh deployment test answer different questions.

The random 60/20/20 split produced 100% accuracy, but consecutive sensor readings are strongly related in time. EMA filtering increases this relationship because each filtered sample contains information from the previous sample.

A random row split can therefore place very similar readings into both training and test sets.

For this reason, the 480/480 result is useful for checking the training and quantization pipeline, but it is probably optimistic as a measure of performance in a different environment.

The separately collected deployment batch gives a more realistic test.

The indoor classes still worked, but the outdoor condition changed enough that the model rejected it.

This is also useful evidence for the reject threshold

$$\tau = 0.12.$$

The threshold was not fully optimized using a dedicated unknown-condition dataset. However, the overcast test shows that UNKNOWN is actually used when a measurement falls outside the learned regions.

5.1 Why the DBSCAN Table is Perfect

The perfect DBSCAN confusion table is a result on the training split, not proof that every future lighting condition will be classified perfectly.

After the logarithmic transformation, the four training conditions formed tight groups.

The three indoor classes were separated mainly along the IR feature, while the sun group was farther away because both UV and IR were much larger.

This explains why DBSCAN matched the training labels exactly but the later overcast outdoor readings did not behave like the original sun measurements.

5.2 KDE and Grid Resolution

The ESP32 does not store the continuous KDE functions.

It stores a

$$16 \times 16$$

sampled version of each surface.

Each scaled feature axis is divided into steps of

$$0.0625.$$

A very small KDE bandwidth can create density detail that is finer than this stored resolution.

This is why the validation likelihood alone should not be used to argue that an arbitrarily small bandwidth would improve the final embedded classifier.

6 Limitations

The classifier represents only four lighting conditions.

Different lamps, brightness levels, sensor angles, distances, rooms, mixed lighting, or weather can move readings outside the original training distribution.

The overcast outdoor batch demonstrated this limitation directly.

The original 60/20/20 test split also came from the same overall recordings as the training data. A stronger future test would split complete recording sessions or time blocks instead of individual rows.

The EMA filter reduces short fluctuations but introduces delay. A sufficiently large or fast change can still move the filtered signal toward another region.

The reject threshold

$$\tau = 0.12$$

was fixed rather than optimized with a dedicated collection of unknown and intermediate lighting conditions.

The TSL2591 also has a finite measurement range. Outdoor operation should be tested under a wider range of sunlight levels to check how the sensor behaves near its upper range.

Finally, the 16×16 grid is a compromise between model size and spatial resolution. Increasing the grid size would preserve more detail but increases storage according to

$$KG^2.$$

7 Project Evidence

The completed hardware performed live classification on the ESP32.

The project repository contains the labelled data, training scripts, generated model, ESP32 firmware, plots, and report:

`https://github.com/AliBaba004/classification-pipeline`

8 Conclusion

A four-class lighting classifier was trained from real UV and IR sensor measurements and deployed to an ESP32.

The original dataset contained 2400 readings.

After logarithmic transformation and scaling, DBSCAN produced four clean training clusters using

$$\varepsilon = 0.02$$

and

$$\texttt{min_samples} = 8.$$

The final KDE model used

$$h = 0.03$$

and was converted into four 16×16 byte lookup tables.

The stored model data requires 1040 bytes, with a measured worst-case quantization error of 0.193%.

The original held-out split produced 480 correct classifications out of 480. A separate 2324-reading

deployment batch produced 100% agreement between the ESP32 and desktop lookup.

LED, fluorescent, and shade remained correct in the fresh batch. The overcast outdoor samples were rejected as UNKNOWN because they did not match the clear-sun training condition.

The filtering tests showed that EMA reduces short variation but also delays the response to a large change.

The isolated model increased flash use by 1060 bytes and did not increase global SRAM. The complete final program used 308800 bytes of flash and 23740 bytes of SRAM.

Classification required approximately 3–6 μs per query.

Overall, the results show that the DBSCAN–KDE pipeline can be reduced to a small lookup model that runs efficiently on the ESP32, while the fresh-batch test also shows why environmental testing outside the original training data is important.